def88edf-a0c7-48c6-8727-17eea0eeb2cc

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Kralizek/AWSSecretsManagerConfigurationExtensions' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/AWSSecretsManagerConfigurationExtensions/AWSSecretsManagerConfigurationExtensions (not in /tmp/inputs).
Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-28T20:04:32.369+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Full 2.0 breaking refresh
</pr_title>

<pr_description>
- [x] Add `object?[]? Args` to `SecretsManagerLogEvent`
- [x] Update `ToLogEventSink` to forward `Args` to `logger.Log()`
- [x] Update private `Log` helper in provider to accept `params object?[]` args (drop `props`)
- [x] Convert all call sites to use message templates with `{Placeholder}` args
- [x] Add regression test: explicit SecretId loading with JSON

flattening roots keys at `GetSecretValueResponse.Name`, not the requested ARN/id
- [x] Replace `HashSet<(string,string)>` loaded-state with `Dictionary<string,string?>` using `InvariantCultureIgnoreCase` so reload comparisons are case-insensitive and consistent with `Data`
- [x] Fix `Dispose()` to capture locals, null fields, cancel, await polling task, then dispose `CancellationTokenSource` in `finally`
- [x] Strengthen `SecretsManagerExtensionsTests` to assert `SecretsManagerConfigurationSource` type and verify `Client`/`AwsOptions` wiring and configure-delegate effects; add internal `Options`/`Client`/`AwsOptions` properties to `SecretsManagerConfigurationSource` for testability
- [x] Document `SecretsManagerConfigurationSource` visibility change (`public` → `internal`) in `MIGRATION.md`
- [x] Enable `<Nullable>enable</Nullable>` in test project
- [x] Build and test (39/39 pass)
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/Kralizek.Extensions.Configuration.AWSSecretsManager/Internal/SecretsManagerConfigurationProvider.cs:93-103</file>
<original_commit>a1f06c8</original_commit>

</comment_thread>

<comment_thread>
<file>src/Kralizek.Extensions.Configuration.AWSSecretsManager/
Internal/SecretsManagerConfigurationProvider.cs:391-394</file>
<original_commit>a1f06c8</original_commit>

</comment_thread>

<comment_thread>
<file>tests/Tests.Extensions.Configuration.AWSSecretsManager/
SecretsManagerExtensionsTests.cs:28-33</file>
<original_commit>a1f06c8</original_commit>

</comment_thread>

<comment_thread>
<file>MIGRATION.md:56-77</file>
<original_commit>a1f06c8</original_commit>

</comment_thread>

</pr_reviews>

<pr_comments>

emain correct across initial load and reload.
   - Keep behavior deterministic.

2. **Dispose `CancellationTokenSource` correctly**
   - Update `Dispose()` in `Internal/
SecretsManagerConfigurationProvider.cs` so the CTS is actually
disposed.
   - Follow the pattern suggested by the review:
     - capture `_cancellationTokenSource` and `_pollingTask`
into locals
     - null out the fields
     - cancel the CTS
     - await the polling task

     - dispose the CTS in `finally`
   - Preserve current cancellation behavior and avoid racey reuse of cleared fields.

3. **Strengthen registration tests**
   - In `tests/Tests.Extensions.Configuration.AWSSecretsManager/SecretsManagerExtensionsTests.cs`, replace weak `It.IsAny<IConfigurationSource>()` assertions with stronger assertions.
   - Verify that the added source is specifically a `SecretsManagerConfigurationSource`.
   - Where applicable, verify that:
     - the expected AWSOptions instance is used
     - the expected `IAmazonSecretsManager` client is used
     - the configure delegate’s effects are reflected in the source/provider options
   - Keep the tests maintainable; do not overfit to incidental internals, but make them strong enough to catch regressions in registration wiring.

4. **Update migration documentation**
   - In `MIGRATION.md`, explicitly call out that `SecretsManagerConfigurationSource` changed from `public` to `internal` in 2.0.
   - Document this as a breaking change for consumers who may have referenced the source type directly.

General requirements:
- Keep nullable annotations correct.
- Keep the code style aligned with the rest of the branch.
- Update or add tests as needed so the changed behavior is covered.
- Do not introduce unrelated refactors.
- After making the changes, summarize exactly how each review comment was addressed.
</comment_old>

<comment_old>
<author>@copilot</author>

> Address the remaining PR review comments on PR #104.
>
> Scope:...

Addressed in bda5669 (main changes in 427a84d):

1. **Loaded-state comparison** — `_loadedValues` is now a

`Dictionary<string,string?>` with `StringComparer.InvariantCultureIgnoreCase`. `FetchConfigurationAsync` and `FetchConfigurationBatchAsync` return the dictionary directly (no HashSet conversion). `SetData` assigns it straight to `Data`. `ReloadAsync` uses a new `DictionaryEquals` helper that uses the dictionary's case-insensitive key comparer.

2. **`Dispose()` fix** — Fields are captured into locals and nulled first, then `Cancel()` is called, the polling task is awaited (catching `OperationCanceledException`), and the CTS is disposed in `finally`.

3. **Stronger registration tests** — `SecretsManagerConfigurationSource` now exposes internal `Options`, `Client`, and `AwsOptions` properties. `SecretsManagerExtensionsTests` captures the added source via `Callback`, asserts it is a `SecretsManagerConfigurationSource`, and verifies that `Client`/`AwsOptions` are the same instances passed in and that configure delegate effects are reflected in `Options`. The test project now has `<Nullable>enable</Nullable>`.

4. **Migration docs** — Added a `SecretsManagerConfigurationSource is now internal` section to `MIGRATION.md` documenting the breaking visibility change and the `AddSecretsManager` migration path.
</comment_old>

```
<comment_new>
<comment_id>4192005110</comment_id>
<author>@Kralizek</author>
Make one final documentation fix in `MIGRATION.md`.

Change the `DuplicateKeyHandling` section so it no longer says that `LastWins` “matches v1.x behaviour”.

Instead, document it accurately:
- `DuplicateKeyHandling` is new in 2.0
- the default is `LastWins`
- this is a behavior change from 1.x, where duplicate keys could cause failures rather than following an explicit precedence policy

Keep the wording short and clear.
</comment_new>
```

</pr_comments>

</comments>

The last **9** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Updated `MIGRATION.md` in 500f58b: the `DuplicateKeyHandling` section heading now reads `DuplicateKeyHandling enum added (behavior change)`, the body explains the enum is new in 2.0, that 1.x had no explicit policy (duplicate keys could cause unpredictable results or failures),

and that the default `LastWins` is an intentional behavior change — with a note to use `Throw` to surface conflicts explicitly.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Kralizek/AWSSecretsManagerConfigurationExtensions' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/AWSSecretsManagerConfigurationExtensions/AWSSecretsManagerConfigurationExtensions (not in /tmp/inputs).
Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-28T20:12:12.289+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Full 2.0 breaking refresh
</pr_title>

<pr_description>
- [x] Add `object?[]? Args` to `SecretsManagerLogEvent`
- [x] Update `ToLogEventSink` to forward `Args` to `logger.Log()`
- [x] Update private `Log` helper in provider to accept `params object?[]` args (drop `props`)
- [x] Convert all call sites to use message templates with `{Placeholder}` args
- [x] Add regression test: explicit SecretId loading with JSON flattening roots keys at `GetSecretValueResponse.Name`, not the requested ARN/id
- [x] Replace `HashSet<(string,string)>` loaded-state with `Dictionary<string,string?>` using `InvariantCultureIgnoreCase`

```
- [x] Fix `Dispose()` to capture locals, null fields, cancel,
await polling task, then dispose `CancellationTokenSource` in
`finally`
- [x] Strengthen `SecretsManagerExtensionsTests` to assert
`SecretsManagerConfigurationSource` type and verify wiring
- [x] Document `SecretsManagerConfigurationSource` visibility
change in `MIGRATION.md`
- [x] Enable `<Nullable>enable</Nullable>` in test project
- [x] Fix `DuplicateKeyHandling` section: document as new in 2.0
and a behavior change from 1.x, not "matching v1.x behaviour"
- [x] Build and test (39/39 pass)
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>









and generated 4 comments.









---

💡 <a>Add Copilot custom instructions</a> for smarter, more
guided reviews. <a href="https://docs.github.com/en/copilot/
customizing-copilot/adding-repository-custom-instructions-for-
github-copilot">Learn how to get started</a>.
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
```

<file>src/Kralizek.Extensions.Configuration.AWSSecretsManager/Internal/SecretsManagerConfigurationProvider.cs:93-103</file>
<original_commit>a1f06c8</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`ReloadAsync` compares `_loadedValues` with `SetEquals`, but `_loadedValues` is a `HashSet<(string,string)>` using the default (case-sensitive) tuple equality, while `Data` is built with `StringComparer.InvariantCultureIgnoreCase`. This can cause unnecessary reloads if key casing differs between loads (even though lookups are intended to be case-insensitive). Consider storing the loaded state in a `Dictionary<string,string?>` with the same comparer as `Data`, or create `_loadedValues` with a custom `IEqualityComparer<(string key,string value)>` that compares `key` using `InvariantCultureIgnoreCase`.
</comment_old>

</comment_thread>

<comment_thread>
<file>src/Kralizek.Extensions.Configuration.AWSSecretsManager/Internal/SecretsManagerConfigurationProvider.cs:391-394</file>
<original_commit>a1f06c8</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`Dispose()` cancels and clears `_cancellationTokenSource` but never disposes it. Since `CancellationTokenSource` implements `IDisposable`, it should be disposed (ideally after the polling task has observed cancellation) to avoid leaking wait handles in long-running processes.
```suggestion
            var cancellationTokenSource = _cancellationTokenSource;
            var pollingTask = _pollingTask;

            _cancellationTokenSource = null;
            _pollingTask = null;

            cancellationTokenSource?.Cancel();
            try { pollingTask?.GetAwaiter().GetResult(); } catch (TaskCanceledException) { }
            finally { cancellationTokenSource?.Dispose(); }
```
</comment_old>

```
</comment_thread>

<comment_thread>
<file>tests/Tests.Extensions.Configuration.AWSSecretsManager/
SecretsManagerExtensionsTests.cs:28-33</file>
<original_commit>a1f06c8</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
These tests only verify that `IConfigurationBuilder.Add()` is
called with *any* `IConfigurationSource`, which is too weak to
catch regressions (e.g., adding the wrong source type or missing
AWS/client wiring). Consider asserting that the added source is
a `SecretsManagerConfigurationSource` and (where applicable)
that it was constructed with the expected client/AWSOptions and
that the options delegate’s effects are reflected in the source/
provider options.
</comment_old>

</comment_thread>

<comment_thread>
<file>MIGRATION.md:56-77</file>
<original_commit>a1f06c8</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`SecretsManagerConfigurationSource` was changed from `public` to
`internal` in 2.0 (see `src/.../Internal/
SecretsManagerConfigurationSource.cs`). If anyone referenced it
directly in 1.x, this is a breaking change and should be called
out explicitly in this migration guide.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@Kralizek</author>
@copilot
Add a regression test for explicit secret id loading with JSON
flattening.

The test should verify that when
```

`SecretsManagerOptions.SecretIds` is used with an ARN/full secret id, the provider still uses the `Name` returned by `GetSecretValueResponse` as the root configuration key for extracted JSON values.

Scenario:

* Configure `SecretsManagerOptions.SecretIds` with a secret ARN/id.
* Mock `GetSecretValueAsync` so the returned `GetSecretValueResponse` has:

  * `ARN` equal to the requested ARN/id
  * `Name` equal to a hierarchical secret name, e.g. `/App/Production/Service/Settings/Nested/Section`
  * `SecretString` containing a generic nested JSON object
* Configure `KeyGenerator` to:

  * strip the known `/App/Production/Service/Settings/` prefix
  * replace `/` with `:`
* Load the provider.
* Assert that nested JSON values are available under the expected configuration keys.

Example secret value:

```json
{
  "Property": "value",
  "Nested": {
    "Enabled": true
  }
}
```

Example expected keys:

```text
Nested:Section:Property
Nested:Section:Nested:Enabled
```

The purpose is to lock down that explicit secret id loading does not root configuration keys at the ARN/id. Keys should be rooted at the actual secret name returned by Secrets Manager.

</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot
> Add a regression test for explicit secret id loading with JSON flattening.
>
...

Added in 8ffa962. The test (`Explicit_secretId_with_json_flattening_roots_keys_at_secret_name_not_arn`) uses a full-form ARN as the SecretId, mocks `GetSecretValueAsync` to return a response with `Name = "/App/Production/Service/Settings/Nested/Section"` and a nested JSON payload, and configures a `KeyGenerator` that strips the `/App/Production/Service/Settings/` prefix and replaces remaining `/` with `:`. It then asserts that `Nested:Section:Property` and `Nested:Section:Nested:Enabled` are available, confirming JSON keys are rooted at the response `Name` rather than the requested ARN.
</comment_old>

<comment_old>
<author>@Kralizek</author>
Address the remaining PR review comments on PR #104.

Scope:
Make only the changes needed to resolve the outstanding review feedback cleanly and consistently with the 2.0 design already in the branch.

Required changes:

1. **Fix loaded-state comparison semantics**
   - In `Internal/SecretsManagerConfigurationProvider.cs`, stop using a case-sensitive `HashSet<(string,string)>` model for `_loadedValues` if reload comparison is meant to follow case-insensitive configuration key semantics.
   - Update the internal loaded-state representation so reload comparisons use the same key comparer as `Data` (`StringComparer.InvariantCultureIgnoreCase`).
   - Prefer a simpler, more direct representation such as a `Dictionary<string,string?>` with the correct comparer, unless there is a strong reason not to.
   - Ensure duplicate key handling semantics remain correct across initial load and reload.

```
   - Keep behavior deterministic.

2. **Dispose `CancellationTokenSource` correctly**
   - Update `Dispose()` in `Internal/
SecretsManagerConfigurationProvider.cs` so the CTS is actually
disposed.
   - Follow the pattern suggested by the review:
     - capture `_cancellationTokenSource` and `_pollingTask`
into locals
     - null out the fields
     - cancel the CTS
     - await the polling task
     - dispose the CTS in `finally`
   - Preserve current cancellation behavior and avoid racey
reuse of cleared fields.

3. **Strengthen registration tests**
   - In `tests/Tests.Extensions.Configuration.AWSSecretsManager/
SecretsManagerExtensionsTests.cs`, replace weak
`It.IsAny<IConfigurationSource>()` assertions with stronger
assertions.
   - Verify that the added source is specifically a
`SecretsManagerConfigurationSource`.
   - Where applicable, verify that:
     - the expected AWSOptions instance is used
     - the expected `IAmazonSecretsManager` client is used
     - the configure delegate's effects are reflected in the
source/provider options
   - Keep the tests maintainable; do not overfit to incidental
internals, but make them strong enough to catch regressions in
registration wiring.

4. **Update migration documentation**
   - In `MIGRATION.md`, explicitly call out that
`SecretsManagerConfigurationSource` changed from `public` to
`internal` in 2.0.
   - Document this as a breaking change for consumers who may
have referenced the source type directly.

General requirements:
- Keep nullable annotations correct.
- Keep the code style aligned with the rest of the branch.
- Update or add tests as needed so the changed behavior is
covered.
- Do not introduce unrelated refactors.
- After making the changes, summarize exactly how each review
comment was addressed.
```

```
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; Address the remaining PR review comments on PR #104.
&gt;
&gt; Scope:...

Addressed in bda5669 (main changes in 427a84d):

1. **Loaded-state comparison** — `_loadedValues` is now a
`Dictionary<string,string?>` with
`StringComparer.InvariantCultureIgnoreCase`.
`FetchConfigurationAsync` and `FetchConfigurationBatchAsync`
return the dictionary directly (no HashSet conversion).
`SetData` assigns it straight to `Data`. `ReloadAsync` uses a
new `DictionaryEquals` helper that uses the dictionary's case-
insensitive key comparer.

2. **`Dispose()` fix** — Fields are captured into locals and
nulled first, then `Cancel()` is called, the polling task is
awaited (catching `OperationCanceledException`), and the CTS is
disposed in `finally`.

3. **Stronger registration tests** —
`SecretsManagerConfigurationSource` now exposes internal
`Options`, `Client`, and `AwsOptions` properties.
`SecretsManagerExtensionsTests` captures the added source via
`Callback`, asserts it is a `SecretsManagerConfigurationSource`,
and verifies that `Client`/`AwsOptions` are the same instances
passed in and that configure delegate effects are reflected in
`Options`. The test project now has `<Nullable>enable</
Nullable>`.

4. **Migration docs** — Added a
`SecretsManagerConfigurationSource is now internal` section to
`MIGRATION.md` documenting the breaking visibility change and
the `AddSecretsManager` migration path.
</comment_old>

<comment_old>
<author>@Kralizek</author>
Make one final documentation fix in `MIGRATION.md`.

Change the `DuplicateKeyHandling` section so it no longer says
that `LastWins` “matches v1.x behaviour”.
```

Instead, document it accurately:
- `DuplicateKeyHandling` is new in 2.0
- the default is `LastWins`
- this is a behavior change from 1.x, where duplicate keys could cause failures rather than following an explicit precedence policy

Keep the wording short and clear.
</comment_old>

<comment_old>
<author>@copilot</author>

> Make one final documentation fix in `MIGRATION.md`.
>
> Change the `DuplicateKeyHandling` section so it no longer says that `LastWins` “matches v1.x behaviour”....

Fixed in 500f58b. The `DuplicateKeyHandling` section now documents it as new in 2.0, states the default is `LastWins`, and explicitly calls out that this is a behavior change from 1.x (where duplicate keys could cause unpredictable results or failures with no explicit policy).
</comment_old>

<comment_new>
<comment_id>4192049412</comment_id>
<author>@Kralizek</author>
Required changes:

1. **Target frameworks**
   - Change the main package TFMs to exactly:
     - `netstandard2.0`
     - `net10.0`
   - Remove `net8.0` / `net9.0` from the main package if still present.
   - Adjust tests/projects/workflows as needed so the repo still builds cleanly with the new target set.

2. **PolySharp**
   - Add **PolySharp** so the project can safely use modern C# constructs while still targeting `netstandard2.0`.
   - Add it in the normal build-time/private-assets way.
   - Do not expose it as a consumer-facing runtime dependency unless strictly required.

3. **README restoration**

```
   - Rework `README.md` so it keeps the cleaner 2.0
structure but restores the useful visibility/trust-building
content from the earlier README.
   - In particular, bring back:
     - community mentions
     - articles/videos/posts using the package
     - similar sections that help establish credibility and
discoverability
   - Do not revert the README blindly; keep the 2.0 API/docs
improvements, but restore the useful social/community
content that was removed.

4. **Migration guide correction**
   - In `MIGRATION.md`, fix the `DuplicateKeyHandling`
section.
   - Do **not** say that `LastWins` matches 1.x behavior.
   - Document it accurately:
     - `DuplicateKeyHandling` is new in 2.0
     - the default is `LastWins`
     - this is a behavior change from 1.x, where duplicate
keys could fail rather than following an explicit precedence
policy

5. **Final repo cleanup**
   - Do one more pass for leftover migration debris and
remove anything that no longer belongs after the 2.0
refresh.
   - Only remove files/config that are genuinely obsolete;
do not delete required compatibility files.

Requirements:
- Keep changes focused on these final adjustments.
- Preserve the current 2.0 API direction.
- Update tests/workflows/docs as needed to stay consistent.
- Summarize exactly what was changed.
</comment_new>
</pr_comments>

</comments>

----
The last **10** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
```

<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.